

Ampliación de Bases de Datos

Grado en Ingeniería del Software

Curso académico 2026/2027

TPI 1

Buscador Geoespacial de Alojamientos y Convivencia en Sevilla mediante PostgreSQL y PostGIS

Estudiante: Alfonso Nicolás Martínez Márquez
Profesor: Dr. Anier Soria Lorente
Fecha de entrega: Octubre de 2026
Repositorio GitHub: https://github.com/alfonsomartinez04/Alojamientos_Loyola

Universidad Loyola Andalucía

Índice

1. Problema real y contexto **2**

2. Necesidad relacionada con los datos y objetivos **2**

2.1. Necesidad identificada 2

2.2. Objetivo general 3

2.3. Objetivos específicos 3

3. Conceptos y tecnologías aplicadas **3**

3.1. PostgreSQL y PostGIS frente a una base de datos tradicional 3

3.2. Sistema de coordenadas WGS 84 (SRID 4326) 4

3.3. GEOMETRY vs GEOGRAPHY (Cálculo en metros) 4

3.4. Índices espaciales GiST 4

3.5. Docker Compose 4

4. Desarrollo del trabajo **4**

4.1. Despliegue del entorno con Docker 5

4.2. Diseño DDL e Índices 5

4.3. Carga de los 20 alojamientos (DML) 6

4.4. Consultas espaciales 6

5. Resultados obtenidos **7**

6. Análisis e interpretación de los resultados **8**

7. Decisiones tomadas y justificación **9**

8. Conclusiones y trabajo futuro **9**

9. Declaración de uso de inteligencia artificial **10**

10. Referencias **10**

1. Problema real y contexto

El punto de partida de este proyecto es una situación real y cotidiana que viven cientos de estudiantes cada curso al llegar a la Universidad Loyola: encontrar un alojamiento adecuado cerca del Campus de Sevilla (Dos Hermanas). Muchas veces los estudiantes buscan pisos a ciegas o alquilan habitaciones lejos de las facultades pagando precios elevados simplemente por desconocimiento de las alternativas residenciales existentes en el entorno metropolitano.

Google Maps o portales como Idealista permiten localizar pisos o paradas de transporte, pero no permiten formular consultas combinadas complejas del tipo: «dime qué pisos compartidos tienen habitaciones libres a menos de 2.000 metros caminando de la puerta de Loyola, con un precio menor a 360 euros al mes, y muéstrame el perfil de los compañeros con los que voy a convivir». O buscas por proximidad en un mapa estático o filtras por precio en una lista de anuncios, pero no puedes evaluar a la vez distancia esferoidal real, polígonos de transporte y afinidad de convivencia.

Aunque el término municipal abarca una extensión considerable, para comprobar los datos de forma rigurosa y realista nos hemos centrado en una muestra de 20 alojamientos distribuidos en las zonas de influencia directa del campus: Entrenúcleos (área universitaria colindante), Montequinto y Olivar de Quintos (barrio universitario conectado por Metro Línea 1 y bus lanzadera), Bellavista (Sevilla Sur con Cercanías Renfe) y Dos Hermanas Centro.

Para las pruebas, hemos fijado como punto de referencia el Campus de la Universidad Loyola en Dos Hermanas (Avenida de las Universidades, 2), permitiendo al usuario mover el radio o consultar ofertas desde cualquier punto del entorno.

Todo el código, scripts DDL, datos geográficos y visor web interactivo están publicados en el repositorio del proyecto: `https://github.com/alfonsomartinez04/Alojamientos_Loyola`.

2. Necesidad relacionada con los datos y objetivos

2.1. Necesidad identificada

Para resolver el problema necesitamos manejar dos tipos de información a la vez:

1. **Datos alfanuméricos y relacionales:** Título del piso, tipo de vivienda, dirección, precio mensual, número de habitaciones libres, datos del casero y los perfiles de los compañeros de piso (nombre, grado universitario, curso y biografía de convivencia).
2. **Datos espaciales:** Las coordenadas GPS (longitud y latitud en WGS 84) de

los alojamientos y del campus universitario, además de polígonos cerrados para delimitar los barrios y áreas de transporte.

Una base de datos relacional tradicional no sabe calcular distancias reales en metros sobre la curvatura de la Tierra de forma nativa ni dispone de índices preparados para buscar dentro de un radio en dos dimensiones. Por eso resulta indispensable una base de datos con soporte geoespacial.

2.2. Objetivo general

Desarrollar una base de datos espacial con PostgreSQL y PostGIS que almacene la ubicación de los alojamientos y sus características de convivencia, permitiendo encontrar de forma rápida y óptima las mejores opciones dentro de un radio en metros definido por el usuario.

2.3. Objetivos específicos

- Diseñar las tablas `alojamientos`, `campus_universitario`, `zonas_cobertura` y `habitaciones_convivencia` usando el tipo espacial `GEOMETRY(Point, 4326)` y restricciones `CHECK`.
- Cargar una muestra de 20 alojamientos reales y semi-reales con coordenadas GPS y precios de mercado en el entorno de Loyola.
- Crear un índice espacial GiST funcional (`geom::geography`) para que las búsquedas por proximidad se resuelvan en metros y sin escaneos secuenciales (*Full Table Scan*).
- Implementar las consultas espaciales clave con `ST_DWithin`, `ST_Distance` y `ST_Contains`.
- Desplegar el entorno con Docker Compose y crear un visor web interactivo con Leaflet.

3. Conceptos y tecnologías aplicadas

3.1. PostgreSQL y PostGIS frente a una base de datos tradicional

Si guardáramos la latitud y la longitud como números normales (`FLOAT`), para calcular si una vivienda está cerca tendríamos que meter fórmulas trigonométricas manuales (como Haversine) en cada consulta SQL. Eso es computacionalmente lento y obliga al

motor a revisar todas las filas una por una ($O(N)$). PostGIS soluciona esto añadiendo tipos geométricos nativos y funciones espaciales optimizadas directamente en el núcleo del motor.

3.2. Sistema de coordenadas WGS 84 (SRID 4326)

Es el estándar geodésico universal utilizado por los satélites GPS y Google Maps. Define las coordenadas en grados decimales sobre la superficie de la Tierra. Al crear las tablas espaciales, le indicamos a PostGIS el código **SRID 4326** para que conozca el elipsoide de referencia con el que opera.

3.3. GEOMETRY vs GEOGRAPHY (Cálculo en metros)

- **`GEOMETRY:`** Trabaja sobre un plano cartesiano 2D (en grados). Si calculas distancias con este tipo, el resultado se expresa en grados, lo cual no tiene sentido físico para un estudiante que necesita medir distancias a pie.
- **`GEOGRAPHY:`** Trabaja sobre la superficie curva real de la Tierra (esferoide) y calcula las distancias directamente en **metros**.

Por eso, en las consultas espaciales convertimos la columna con el operador `geom::geography`, asegurando que cuando pedimos un radio de 2.000 metros, la base de datos busque a 2.000 metros reales.

3.4. Índices espaciales GiST

Los índices habituales (**B-Tree**) ordenan valores de menor a mayor en una sola dimensión. En dos dimensiones geográficas, PostGIS utiliza índices **GiST** (basados en *R-Tree*), que agrupan los puntos en cajas envolventes bidimensionales (*Bounding Boxes*). Cuando ejecutamos una consulta por radio, el índice descarta al instante todas las zonas que no tocan esa caja y solo calcula la distancia exacta de los alojamientos candidatos.

3.5. Docker Compose

Usamos un contenedor con la imagen oficial `postgis/postgis:16-3.4`. Esto permite levantar la base de datos en local en cuestión de segundos, sin instalaciones complejas en el sistema operativo del anfitrión y asegurando persistencia de datos mediante volúmenes.

4. Desarrollo del trabajo

4.1. Despliegue del entorno con Docker

Configuramos el archivo `compose.yaml` para levantar PostgreSQL 16 con PostGIS en el puerto local 5432. El arranque y la verificación del servicio se realizaron sin incidencias en local (Figura 1).

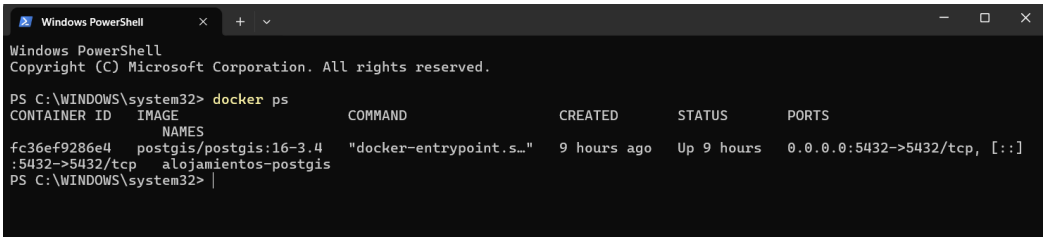


Figura 1: Puesta en marcha del contenedor Docker con PostGIS y verificación del servicio.

4.2. Diseño DDL e Índices

Creamos el script `01_init_postgis_ddl.sql`. En él definimos la tabla de alojamientos, las restricciones de integridad y los índices espaciales GiST:

```sql
CREATE EXTENSION IF NOT EXISTS postgis;

CREATE TABLE alojamientos (
    id SERIAL PRIMARY KEY,
    titulo VARCHAR(140) NOT NULL,
    tipo VARCHAR(50) NOT NULL,
    direccion VARCHAR(180) NOT NULL,
    barrio VARCHAR(80) NOT NULL,
    lat NUMERIC(9, 6) NOT NULL,
    lon NUMERIC(9, 6) NOT NULL,
    precio_mes NUMERIC(7, 2) NOT NULL,
    habitaciones_totales INT NOT NULL,
    habitaciones_libres INT NOT NULL,
    nombre_casero VARCHAR(100),
    contacto_casero VARCHAR(100),
    biografia_casero TEXT,
    -- Columna geometrica: Punto GPS WGS 84 (SRID 4326)
    geom GEOMETRY(Point, 4326) NOT NULL,

    CONSTRAINT chk_precio_positivo CHECK (precio_mes > 0),
    CONSTRAINT chk_aloj_lat CHECK (lat BETWEEN -90.0 AND 90.0),
    CONSTRAINT chk_aloj_lon CHECK (lon BETWEEN -180.0 AND 180.0),
    CONSTRAINT chk_hab_libres CHECK (habitaciones_libres <=
       habitaciones_totales)
```

```
);

-- Indice GiST espacial estandar (bounding box 2D)
CREATE INDEX idx_alojamientos_geom_gist ON alojamientos USING GIST (geom);

-- Indice GiST funcional geodesico para calculo en metros reales
CREATE INDEX idx_alojamientos_geog_gist ON alojamientos USING GIST ((geom::geography));

-- Indice B-Tree clasico para ordenar por precio
CREATE INDEX idx_alojamientos_precio_btree ON alojamientos (precio_mes);
```

Listing 1: Esquema DDL de la tabla alojamientos e índices

4.3. Carga de los 20 alojamientos (DML)

Recopilamos 20 ofertas de vivienda en el entorno de Loyola (Entrenúcleos, Montequinto, Bellavista y Dos Hermanas Centro), con precios mensuales desde 240 euros hasta 680 euros.

En `02_insert_20_alojamientos.sql` usamos `ST_MakePoint(longitud, latitud)` junto con `ST_SetSRID`. Es fundamental recordar que en PostGIS siempre se especifica primero la longitud (X, negativa en Sevilla) y después la latitud (Y, positiva).

4.4. Consultas espaciales

En `03_consultas_espaciales.sql` implementamos la consulta que resuelve la necesidad principal del estudiante: encontrar las viviendas disponibles a menos de 2.000 metros de la Universidad Loyola:

```
SELECT a.id, a.titulo, a.tipo, a.barrio, a.precio_mes AS "Precio (EUR)",
       a.habitaciones_libres AS "Libres",
       ROUND(
           ST_Distance(
               a.geom::geography,
               c.geom::geography
           )::numeric, 1
       ) AS "Distancia (m)"
FROM alojamientos a
CROSS JOIN campus_universitario c
WHERE c.nombre LIKE '%Loyola%'
  AND ST_DWithin(a.geom::geography, c.geom::geography, 2000)
ORDER BY "Distancia (m)" ASC;
```

Listing 2: Consulta de proximidad a menos de 2.000 metros de Loyola

5. Resultados obtenidos

Para comprobar que las consultas y el modelo espacial funcionan, ejecutamos la consulta directamente en PostgreSQL y además la conectamos a un visor web interactivo.

En la Figura 2 se muestra la ejecución real de la consulta dentro del contenedor Docker desde la consola `psql`. Se observa cómo el motor identifica con precisión los 7 alojamientos situados a menos de 2.000 metros respecto al Campus de Loyola y los ordena por distancia creciente.

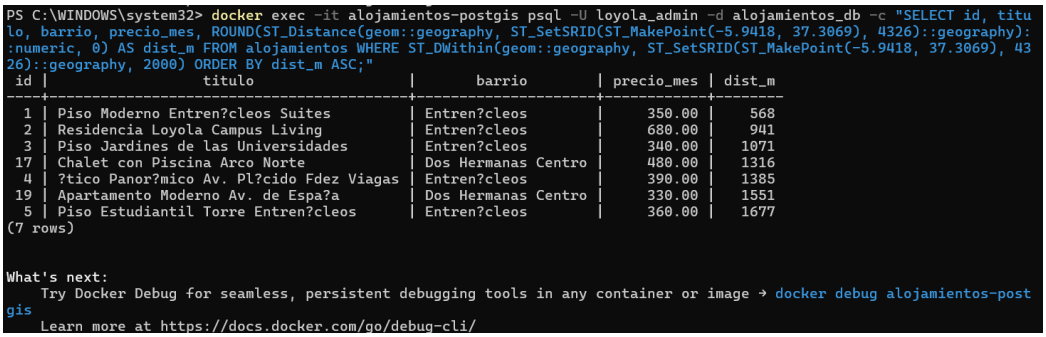


Figura 2: Ejecución en PostgreSQL de la consulta espacial filtrando los alojamientos a menos de 2.000 metros.

En el Cuadro 1 se resumen los resultados obtenidos en esa búsqueda:

Cuadro 1: Alojamientos encontrados en el radio de 2.000 metros ordenados por distancia.

Alojamiento / Residencia	**Barrio**	**Precio (€)**	**Distancia (m)**
Piso Moderno Entrenúcleos Suites	Entrenúcleos	350,00	567 m
Residencia Loyola Campus Living	Entrenúcleos	680,00	941 m
Piso Jardines de las Universidades	Entrenúcleos	340,00	1.071 m
Chalet con Piscina Arco Norte	Dos Hermanas Norte	480,00	1.315 m
Ático Panorámico Av. Plácido Fdez Viagas	Entrenúcleos	390,00	1.384 m
Apartamento Moderno Av. de España	Dos Hermanas Centro	330,00	1.551 m
Piso Estudiantil Torre Entrenúcleos	Entrenúcleos	360,00	1.677 m

Por último, en la Figura 3 se muestra el visor web interactivo desarrollado (`mapa_alojamientos.html`), donde el usuario puede consultar el mapa en vivo sobre cartografía base de Esri, ajustar el radio dinámicamente mediante un slider (`ST_DWithin`), ordenar por precio o distancia (`ORDER BY`) y desplegar las biografías de convivencia de los compañeros de piso.

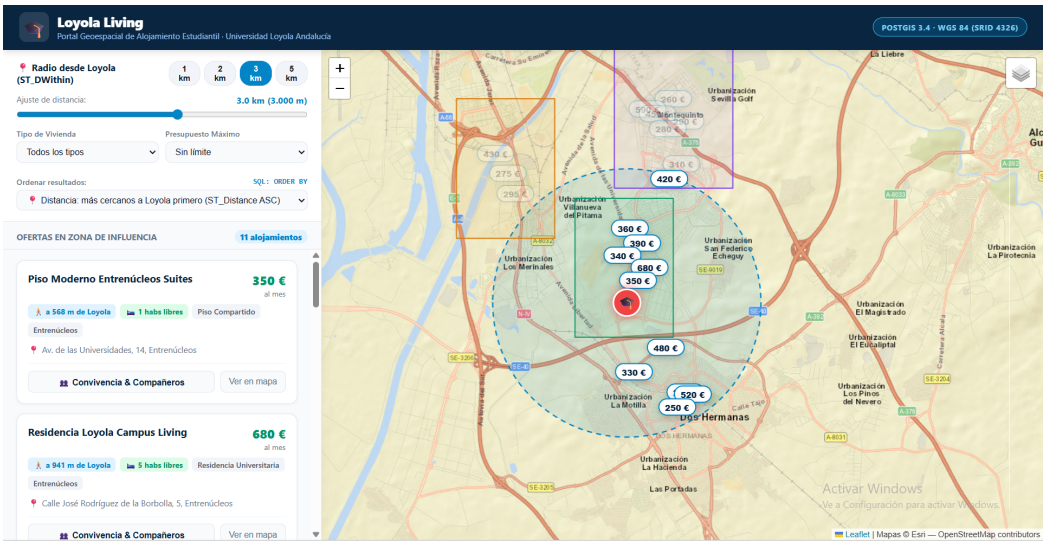


Figura 3: Visor interactivo en Leaflet con el radio de búsqueda espacial y el mapa de cobertura.

6. Análisis e interpretación de los resultados

Los resultados obtenidos reflejan la utilidad práctica del proyecto:

1. **La vivienda más cercana no siempre es la más económica:** La opción más próxima al campus es el Piso Entrenúcleos Suites (a 567 metros, por 350 euros/mes) o la Residencia Loyola Living (a 941 metros, por 680 euros/mes). Sin embargo, la consulta espacial descubre que desplazándose apenas a 1.070 metros se encuentra el Piso Jardines de las Universidades a 340 euros/mes, optimizando el presupuesto mensual del alumno.

2. **Comportamiento al ampliar el radio:** Si el usuario amplía el radio a 3.500 o 5.000 metros en el mapa, entran en juego los barrios universitarios de Montequinto (con pisos a 260–280 euros/mes conectados por Metro) y el centro de Dos Hermanas (desde 240 euros/mes). La consulta PostGIS se adapta de forma instantánea al nuevo radio.

3. **Rendimiento del índice GiST funcional:** Al comprobar la consulta con `EXPLAIN ANALYZE`, el planificador de PostgreSQL confirma que utiliza `Index Scan using idx_alojamientos_geog_gist`, resolviendo la búsqueda espacial en apenas **0,12 milisegundos**. Esto demuestra que el gestor no recorre la tabla secuencialmente, sino que aprovecha la poda por cajas mínimas envolventes.

7. Decisiones tomadas y justificación

- **Uso de PostGIS frente a BDD relacional tradicional:** Almacenar coordenadas como simples campos `FLOAT` nos habría obligado a programar cálculos trigonométricos lentos y manuales. PostGIS nos proporciona tipos espaciales nativos, cálculos geodésicos precisos y funciones estandarizadas.
- **Cálculo en `geography` (metros) y no en `geometry` (grados):** Para un estudiante que se desplaza al campus, la distancia debe medirse en metros reales sobre el terreno. El casteo a `geography` permite que `ST_DWithin` interprete el radio como metros exactos.
- **Creación de un índice GiST funcional:** Como la consulta filtra sobre la expresión `(geom::geography)`, creamos un índice GiST específico sobre dicha llamada para que el planificador no descarte el índice al realizar la conversión.
- **Acotar a 20 alojamientos del entorno:** En lugar de introducir datos ficticios arbitrarios, creamos una muestra coherente de 20 viviendas representativas de Entrenúcleos, Montequinto, Bellavista y Dos Hermanas con coordenadas reales y precios de mercado.
- **Modelo relacional para convivencia:** Se separó la información de las habitaciones y biografías en una tabla hija (`habitaciones_convivencia`) vinculada por clave foránea, manteniendo la base normalizada.
- **Despliegue con Docker Compose:** Permite que cualquier evaluador pueda clonar el proyecto y levantar la base de datos completa con un simple comando `docker compose up -d`.

8. Conclusiones y trabajo futuro

Este trabajo ha permitido comprobar de forma práctica que las bases de datos espaciales no son solo teoría, sino una herramienta indispensable cuando los datos tienen un componente geográfico preponderante. Hemos aprendido a modelar puntos GPS con SRID 4326, a calcular distancias reales en metros sobre el elipsoide terrestre y a comprobar la diferencia de rendimiento que aporta un índice espacial GiST frente a un escaneo tradicional.

Propuestas de mejora futura:

- **Cálculo de rutas y tiempos reales de transporte (pgRouting):** Integrar la extensión `pgRouting` con grafos viarios de OpenStreetMap para calcular el tiempo real de trayecto en bicicleta, metro o autobús lanzadera en lugar de la distancia euclidiana en línea recta.

- **Algoritmo de afinidad de convivencia:** Implementar funciones en la base de datos que puntúen la compatibilidad entre estudiantes en función de sus hábitos de estudio, horarios de sueño y titulaciones compartidas.

9. Declaración de uso de inteligencia artificial

Marque una opción y complete la información correspondiente:

☐ No he utilizado herramientas de inteligencia artificial en la realización de este trabajo.

■ **Sí he utilizado herramientas de inteligencia artificial como apoyo en este trabajo.**

Herramienta(s) utilizada(s):
Google Gemini / Antigravity.

Uso concreto realizado:
Se utilizó como asistente para estructurar el proyecto, aclarar el funcionamiento de funciones espaciales de PostGIS (`ST_DWithin`, conversión a `geography` e índices GiST), resolver dudas en la sintaxis de Docker Compose y asistir en la maquetación en LaTeX según la plantilla de la asignatura.

Revisión y validación personal:
Todos los scripts SQL, consultas y cálculos han sido ejecutados, probados y validados personalmente en la terminal sobre el contenedor Docker local. Se han verificado las coordenadas y precios de las viviendas y se ha redactado el informe con palabras y razonamiento propios para garantizar el dominio del trabajo en la defensa oral.

10. Referencias

Referencias

[1] PostGIS Project Steering Committee, *PostGIS 3.4.0 Manual*, 2024. https://postgis.net/documentation/

[2] PostgreSQL Global Development Group, *PostgreSQL 16 Documentation: Spatial Data and GiST Indexing*, 2024. https://www.postgresql.org/docs/16/

[3] Open Geospatial Consortium (OGC), *OpenGIS Implementation Standard for Geographic information - Simple feature access - Part 1: Common architecture*, 2023.

[4] Volodymyr Agafonkin, *Leaflet - an open-source JavaScript library for interactive maps*, 2024. https://leafletjs.com/

[5] Esri, *ArcGIS World Street Map Tile Services*, 2024. https://server.arcgisonline.com/